

[Regional SPX] Hazard Reporting

- 1. Project Background
- 2. Implementation Details
 - 2.1. User Flow
 - 2.2. Agent Overview
 - 2.3. Learnings
 - 2.3.1. Model Comparison for Common Image Processing Scenarios
 - 2.3.1.1. Text Extraction (OCR)
 - 2.3.1.2. Handwriting Recognition
 - 2.3.1.3. Image Understanding & VQA
 - 2.3.1.4. Document Layout & Form Parsing
 - 2.3.2. Pass Image to Tools Around Agents in Multi-Agent Design
 - 2.3.3. Configure Structure Output
 - 2.3.4. Google OAuth 2 Config

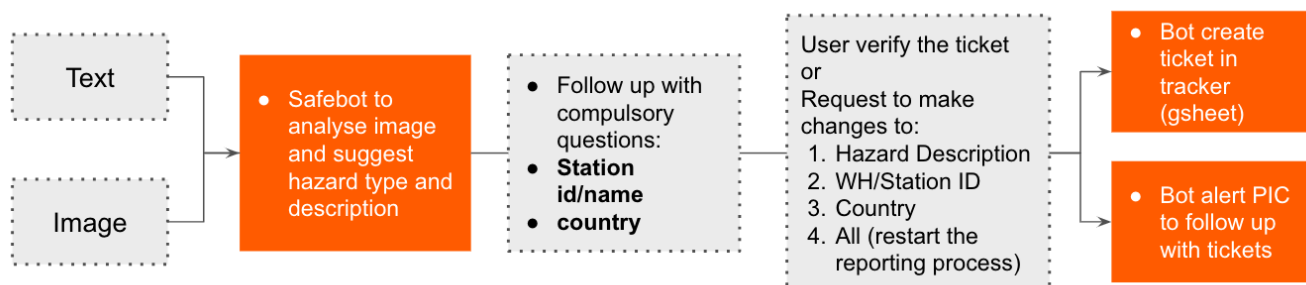
1. Project Background

Team	Regional SPX
Main Contributor	kelly.tay
Target End Users	Local warehouse staffs?
Objective	To allow local wh workers and staffs to report potential hazard in the workplace via seataalk
Expected Outreach	Near future plans would be to roll out this bot to all whs/hubs across 6 markets (1000++ locations). On average per stations would report 1-2 hazards weekly.

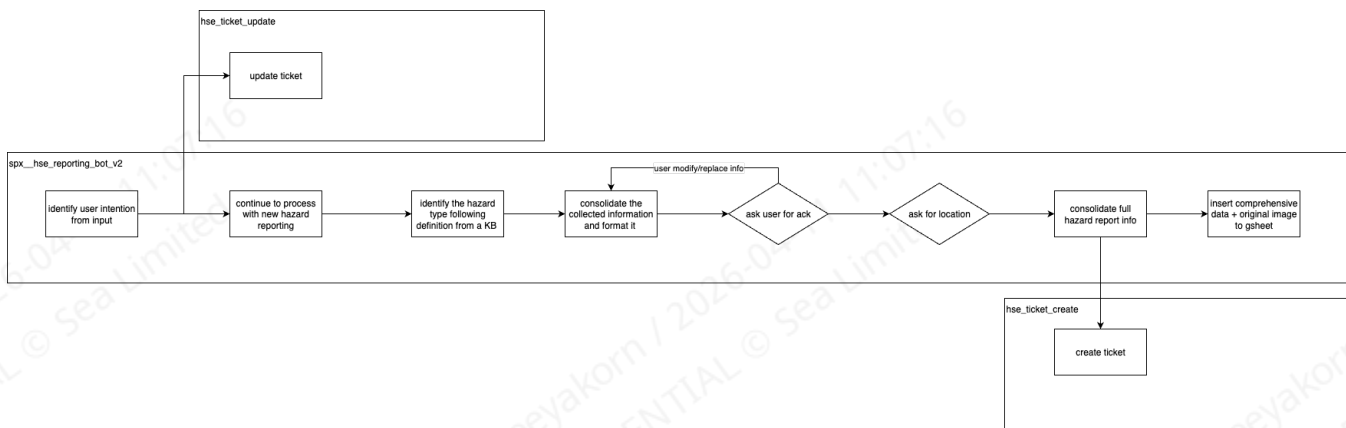
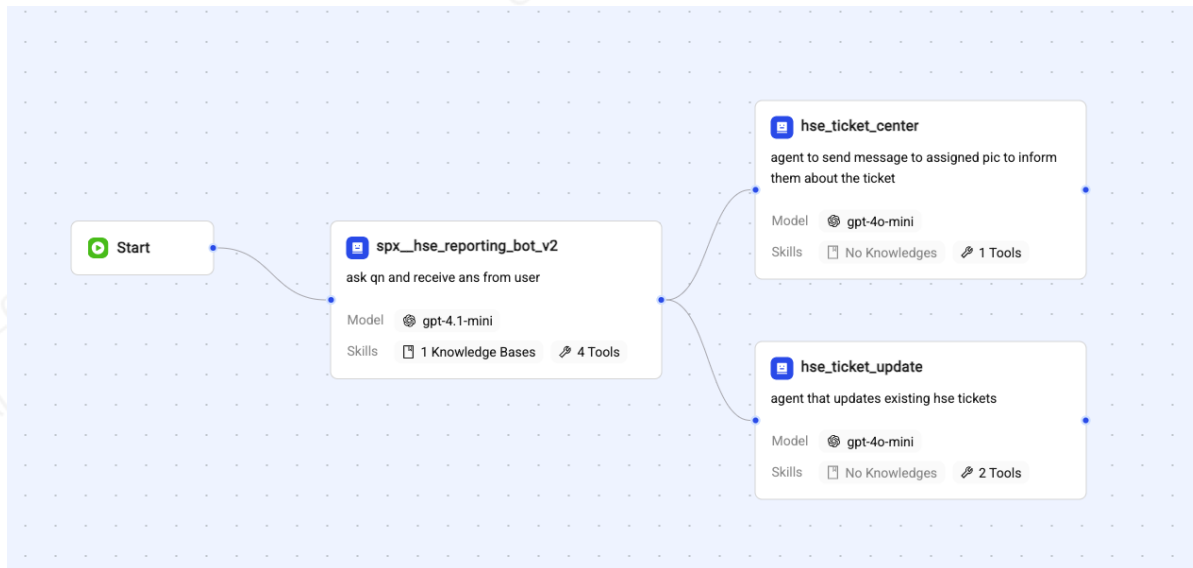
2. Implementation Details

2.1. User Flow

HSE Safebot User Flow



2.2. Agent Overview



2.3. Learnings

2.3.1. Model Comparison for Common Image Processing Scenarios

Scenarios	Description	Top-tier	Mid-tier	Lightweight
2.3.1.1. Text Extraction (OCR)	Extract printed text from images (e.g. scanned documents, street signs).	- GPT-5 / Gemini 2.5 Pro / Claude 4 Sonnet - Accuracy: $\geq 98\%$ on high-resolution scans - Handles complex layouts (multi-column, tables) - Cost: High; longer inference time	- GPT-4.1 / GPT-4o / Qwen2.5-VL-72B - Accuracy: $\sim 94-97\%$ on standard documents - Good at mixed text/images, simple tables - Cost: Moderate; reasonable throughput	- GPT-5 Nano / GPT-4.1 Nano / GPT-o3-o4 Mini / Qwen2.5-VL-7B / Claude 3 Haiku - Accuracy: $\sim 85-92\%$, drops on complex layouts - Best for high-volume batch jobs where some errors are tolerable - Cost: Low; high throughput

2.3.1.2. Handwriting Recognition	Interpret cursive or printed handwriting (notes, forms).	• GPT-5 / Gemini 2.5 Pro • Learns diverse handwriting styles (even low-contrast) • Contextual correction (e.g. “St.” vs. “St.”) • Cost: High; inference latency up to 1–2 s per image	• GPT-4.1 Mini / Claude 3.7 Sonnet • Good on neat handwriting, common dictionaries • Struggles with very stylized scripts or heavy noise • Cost: Moderate; ~ 0.5 s/image	• GPT-5 Nano / GPT-4.1 Nano / Claude 3 Haiku • Basic recognition—works best on clearly written text • Limited contextual correction; higher error rate on smudged text • Cost: Low; < 0.2 s/image
2.3.1.3. Image Understanding & VQA	Scene description, object detection, visual question answering.	• GPT-5 / Claude 4 Sonnet / Gemini 2.5 Pro • Rich scene graphs, fine-grained object attributes (“red striped awning”) • Strong commonsense reasoning (“Why is the person wearing a helmet?”) • Cost: High; best for mission-critical insights	• GPT-4o / Qwen2.5-VL-72B / Claude 3.7 Sonnet • Robust object detection and basic reasoning • Good for e-commerce catalogs, asset tagging • Cost: Moderate; high throughput for typical enterprise apps	• GPT-4o Mini / GPT-5 Mini / Qwen2.5-VL-7B / Claude 3.5 Haiku • Decent at identifying common objects, simple captions • Limited depth on abstract reasoning or rare object classes • Cost: Low; ideal for real-time mobile/edge agents
2.3.1.4. Document Layout & Form Parsing	Identify fields, checkboxes, tables, and logical grouping.	• GPT-5 / Gemini 2.5 Pro • Auto-detects form structures, nested tables, signature fields • Integrates with downstream workflows (e-signing, ERP) • Cost: High; complex pre- and post-processing	• GPT-4.1 / Qwen2.5-VL-72B • Handles standard forms, invoices, receipts • May require manual template tuning for exotic layouts • Cost: Moderate	• GPT-5 Nano / GPT-4.1 Nano / Claude 3 Haiku • Best for single-column forms or simple tables • Limited robustness on multi-column or handwritten fields • Cost: Low; plug-and-play

2.3.2. Pass Image to Tools Around Agents in Multi-Agent Design

In the case from SPX, they're trying to store the reported image together with processed data to google sheets as a record for an easy and consolidated reference. User input images can be injected to tools by Smart platform with the argument name `smart_human_input_image_attachments`

Refer to this guide for more details: [SMART Tool FAQ#HowtogetHumanImageAttachmentInputfromtools?\(new\)](#)

2.3.3. Configure Structure Output

In the case from SPX, their agent will call tool to create ticket. The ticket creation requires a standardise and fix format. Currently, users who need an agent to return responses in a specific structured format (e.g., JSON) must rely on prompt instructions. Example:

```
{
  "GRASS_DATE_LOWER_BOUND": "YYYY-MM-DD",
  "GRASS_DATE_UPPER_BOUND": "YYYY-MM-DD",
  "GRASS_HOUR_LOWER_BOUND": INT,
  "GRASS_HOUR_UPPER_BOUND": INT,
  "GRASS_REGION": <grass_region>,
  "WHS_ID": <whs_id>,
  "intent": ["PRODUCTIVITY_SUMMARY", "BOTTOM_OPERATORS"],
}
```

This method is **unreliable and brittle**. The LLM may not consistently follow the format, leading to:

- Format inconsistencies
- Errors when responses are consumed by downstream systems (e.g., APIs, chained agents)

Smart provides this [Structure Output](#) function to **enforce schema-based output** by allowing users to define an OpenAPI-compliant schema. With this:

- Agents will use function-calling/JSON-mode capabilities of advanced LLMs (e.g., OpenAI, Gemini)
- Responses will be **guaranteed to follow a defined, machine-readable format**
- It enables **reliable and scalable** multi-agent workflows

Skills

Tools



Add Tools to allow your agent expand the capabilities, such as web searching.

Knowledges



Add knowledge to empower your Agent with the domain expertise.

Structured Output ?



Enter an [OpenAPI schema object](#) to constrain the model output

1 Press Tab to use this example:



```
{
  "type": "object",
  "properties": {
    "response": {
      "type": "string"
    }
  }
}
```

Conversational Experience

> Early Response ?



> Collect Feedback in SeaTalk ?



Session Controls

Private Mode ?



2.3.4. Google OAuth 2 Config

[How to setup OAuth 2 for your own Python script](#)